

Shell Scripting: File Handling & Automation

(Reading/Writing Files, grep, find, cron, at)

File Handling and Automation

- Shell scripting is the primary mechanism for File Handling and Automation in Linux/Unix systems. It allows users to execute sequences of commands and logic to manage system tasks, process data, and schedule operations efficiently without direct human intervention.

1. File Handling
2. File and Content Searching
3. Automation and Job Scheduling

1. File Handling

- File handling involves reading, writing, and manipulating files from within a shell script. This is fundamental for managing configuration, processing logs, and structuring data workflows.

1. Reading Files Line by Line
2. Writing to Files (Redirection)

1.1 Reading Files Line by Line

- The standard method for reading a file uses a while loop combined with the read command and input redirection (<). This approach is crucial because it safely processes files without issues caused by spaces, tabs, or special characters in filenames or content.
- Key Techniques:
 - Input Field Separator (IFS): Setting IFS= prevents leading/trailing whitespace from being unintentionally stripped from lines.
 - Raw Read (-r): The -r flag prevents backslash characters from being interpreted as escape sequences, ensuring the line is read exactly as it appears in the file.

1.1 Reading Files Line by Line

- The standard approach uses a while loop to safely process file content, assigning each line to a variable.

Command syntax : `while IFS= read -r line; do ... done < filename`

One-Line Illustration :

`while IFS= read -r line; do echo "Got: $line"; done < data.txt`

Explanation : This robust structure reads data.txt line by line. The echo command inside the loop executes for every line found in the file.

1.2 Writing to Files (Redirection)

- Output redirection is the core of writing files. Commands (like echo or cat) generate output, which is then directed away from the standard output (the terminal) into a file.
- Overwriting (>): The single greater-than sign overwrites a file. If the file exists, its contents are completely deleted before the new data is written. If the file does not exist, it is created.
- Appending (>>): The double greater-than sign appends data. New data is added to the end of the existing file, preserving the previous content. This is essential for logging and historical data tracking.
- Here Documents (<< EOF): This is a specialized redirection method used to provide multiple lines of input (often configuration or report text) directly within the script to a command, typically cat, which then writes the block to a file.

1.2 Writing to Files (Redirection)

- Writing uses redirection operators (> and >>) to manage output flow into a file.

Overwriting (New File/Clear)

- **Operator** : > (Single redirect) - ERASES existing file content.
- **One-Line Illustration** :

```
echo "New session started." > log.txt
```

1.2 Writing to Files (Redirection)

Appending (Adding Content)

- **Operator :** >> (Double redirect) - ADDS content to the end of the file.
- **One-Line Illustration :**

```
echo "Task finished successfully." >> log.txt
```


1.2 Writing to Files (Redirection)

Multi-Line Write

- **Operator** : `cat << EOF > filename`
- **One-Line Illustration** :

```
cat << END_MSG > notes.txt; echo "Finished writing."
```

2. File and Content Searching

- Linux provides specialized tools for locating files and searching their contents efficiently

1. Content Searching with `grep`
2. File Locating with `find`

2.1 Content Searching with grep

- The grep (Global Regular Expression Print) utility searches for lines containing a specified pattern within a file or stream. It is the workhorse for log analysis and data filtering.

Mechanism: grep uses Regular Expressions (Regex) to define complex search patterns.

- Key Functions: It can be used to perform case-insensitive searches (-i), count matches (-c), print lines that do not match the pattern (-v), and search recursively across directories (-r).

2.1 Content Searching with grep

Element	Content
Option: -i	Case-Insensitive match.
Illustration	<code>grep -i "warning" server.log</code>
Option: -v	Invert Match (shows lines that do not match).
Illustration	<code>grep -v "^#" config.ini</code> (Filters out commented lines)
Option: -r	Recursive search through directories.
Illustration	<code>grep -r -l "TODO" ~/source_code</code> (Lists files with "TODO")

2.1 Content Searching with grep

Command Option	Description	Advanced Example
-E or egrep	Enables Extended Regular Expressions (ERE), supporting metacharacters like +, ?, , and ()`.	
-w	Matches the pattern only as a whole word.	grep -w "user" names.txt (Matches "user" but not "users" or "username").
-l	Prints only the names of the files that contain the matching pattern, not the lines themselves.	grep -l "secret key" *.conf (Identifies all configuration files containing "secret key").
-c	Prints a count of matching lines instead of the lines themselves.	grep -c "^#" config.ini (Counts the number of commented lines in the file).
-A N / -B N / -C N	Prints Context lines. -A is After, -B is Before, and -C is Context (both before and after).	grep -A 3 "Failure at line 42" debug.log (Shows the failure line plus the 3 lines immediately following it).

2.2 Locating Files with find

- The find command searches the directory hierarchy for files based on their metadata or attributes, not their content.
- Search Criteria: find can filter by name (-name), type (-type f for file, -type d for directory), size (-size), modification time (-mtime), and permissions (-perm).
- Execution (-exec): Its most powerful feature is the ability to execute an arbitrary command on every item it successfully locates, enabling batch operations like deletion, renaming, or changing permissions.

2.2 Locating Files with find

- find locates files based on attributes (name, size, time) and can execute commands on the results.

Concept	Criteria/Option	One-Line Illustration
Find by Name	find PATH -name PATTERN	<code>find . -name "*.bak"</code>
Find by Age (Older than 7 days)	find PATH -mtime +N	<code>find /tmp -type f -mtime +7</code>
Find and Delete	find ... -delete	<code>find /var/log -name "*.old" -delete</code>
Find and Execute	find ... -exec COMMAND {} \;	<code>find . -name "*.sh" -exec chmod +x {} \;</code>

3. Automation and Job Scheduling

- Automation involves scheduling scripts or commands to execute at specific times or intervals, ensuring system maintenance and routine tasks run reliably.

1. Recurring Jobs with `cron`
2. Single-Run Jobs with `at`

3.1 Recurring Jobs with cron

cron is the primary service for scheduling tasks that need to run repeatedly and reliably (e.g., daily backups, hourly system checks).

- ❑ Mechanism: Jobs are defined in the crontab (cron table) file. Each entry consists of six fields defining the schedule, followed by the command to execute.
- ❑ Crontab Fields: They represent the Minute, Hour, Day of Month, Month, and Day of Week, followed by the command. Special characters like the asterisk (*) for every value) and the slash (/ for step values, e.g., every 15 minutes) allow for complex scheduling.
- ❑ Environment: Cron jobs execute in a minimal shell environment, making it mandatory to use absolute paths for all commands and scripts to ensure they execute correctly.

3.1 Recurring Jobs with cron

- cron manages tasks that need to run repeatedly. You edit the crontab file to add scheduled entries.

Concept	Command Structure	Crontab One-Line Illustration (Entry)
Edit Crontab	crontab -e	N/A (opens an editor for the file)
List Jobs	crontab -l	crontab -l
Run Daily at 4 AM	MIN HOUR DOM MON DOW COMMAND	0 4 * * * /usr/local/bin/daily_report.sh
Run Every 30 Minutes	*/30 * * * * COMMAND	*/30 * * * * /usr/bin/check_ping.sh
Explanation	Crontab entries are added directly to the file opened by crontab -e. The entry for 4 AM runs when the minute is 0 and the hour is 4, every day of the month/week/year (* * *).	

3.2 Single-Run Jobs with at

- The at utility schedules a command or script to run once at a specific time in the future. It is ideal for non-recurring tasks.
- Mechanism: The user specifies the time (which can be absolute, like 14:30, or relative, like now + 3 hours) and then inputs the command(s). The system executes the job when the specified time is reached.
- Management: Jobs can be viewed using atq and removed using atrm.

3.2 Single-Run Jobs with at

- The at command runs a command or script only once at a specified time. Commands are piped or entered interactively, but can be done in one line for scripting.

Concept	Command Structure	One-Line Illustration
Schedule a Job	echo "COMMAND" at TIME	echo "/home/user/cleanup.sh" at 03:00 tomorrow
View Pending Jobs	atq	atq
Remove a Job	atrm JOB_NUMBER	atrm 5
Explanation	The echo sends the command string as standard input (`	) to the at` utility, scheduling it for 3:00 AM the next day.

THANK YOU!!!!